

TP2 - Programmation 2 - Gr. 432 - Projet créatures

Maxime Flageole

1 juin 2026

Énoncé

Dans ce deuxième TP de programmation orientée objet, vous devrez réaliser un système de gestion et de combat de créatures en console.

Ce travail teste non seulement votre compréhension des concepts de programmation orientée objet, mais également votre capacité à concevoir une architecture logicielle claire et réutilisable.

Ce travail doit être réalisé en équipe de deux étudiants.

L'utilisation de collections, de l'héritage, du polymorphisme, des méthodes virtuelles et des constructeurs/destructeurs avec héritage est obligatoire.

Création de créatures

Votre première tâche consiste à permettre à l'utilisateur de créer des créatures personnalisées.

Chaque créature possède au minimum :

- Un nom
- Un type
- Des points de vie
- Une valeur d'attaque
- Une valeur de défense

Les statistiques sont distribuées selon un système de score.

Chaque créature dispose de **100 points à distribuer** entre ses statistiques selon les règles suivantes :

- 1 point investi en attaque = 1 point d'attaque
- 1 point investi en défense = 1 point de défense
- 1 point investi en vie = 5 points de vie

Chaque statistique doit recevoir au moins 1 point. Une créature n'est pas valide si cette condition n'est pas respectée.

Vous devez permettre la création d'au moins quatre types de créatures différents.

Par exemple :

- Feu
- Eau
- Plante
- Électrique

Chaque type doit être représenté par sa propre classe dérivée.

Les attaques d'une créature sont entièrement déterminées par son type (toutes les créatures d'un même type possèdent la même liste d'attaques).

Utilisez l'héritage pour cette partie.

Chaque type possède également un effet passif unique :

- Feu : inflige 10% de dégâts supplémentaires.
- Eau : réduit tous les dégâts reçus de 10%.
- Plante : régénère 5% de ses points de vie à la fin de chaque tour.
- Électrique : possède 20% de chances de répéter son attaque une seconde fois.

Gestion des créatures

Toutes les créatures créées doivent être conservées dans une collection.

L'utilisateur doit pouvoir :

- Afficher toutes les créatures existantes
- Consulter les statistiques d'une créature
- Modifier une créature existante
- Supprimer une créature

Lorsqu'une créature est supprimée, elle doit être retirée correctement de la collection.

Partie facultative : sauvegarde des créatures

Aucun point supplémentaire n'est attribué à cette partie, mais un *challenge* intéressant consiste à créer un système de chargement et de sauvegarde des créatures.

Au lancement du programme, vos créatures sont chargées à partir d'un fichier. Lors d'une sauvegarde, la liste des créatures présentes dans le programme est enregistrée dans ce fichier.

Cela vous permet de conserver les modifications apportées à votre collection de créatures entre les exécutions du programme.

Types et attaques

Chaque type de créature possède exactement quatre attaques.

Les attaques d'un type doivent être différentes de celles des autres types.

Chaque attaque possède au minimum :

- Un nom
- Un multiplicateur de dégâts

Par exemple :

- Attaque rapide : multiplicateur 0.8
- Attaque normale : multiplicateur 1.0
- Attaque puissante : multiplicateur 1.2
- Attaque ultime : multiplicateur 1.5

Les noms exacts des attaques sont laissés à votre discrétion.

Les attaques sont calculées selon l'équation suivante :

$$[D = A \times M \times \frac{100}{100+F}]$$

où :

- D représente les dégâts infligés ;
- A représente la statistique d'attaque de la créature attaquante ;
- M représente le multiplicateur de dégâts de l'attaque utilisée ;
- F représente la statistique de défense de la créature ciblée.

Les dégâts doivent être arrondis à l'entier le plus près.

Sélection des combattants

Une fois des créatures créées, l'utilisateur doit pouvoir démarrer un combat.

Deux créatures existantes sont alors sélectionnées parmi la collection.

Les statistiques complètes des deux combattants doivent être affichées avant le début du combat.

Combat

Le combat se déroule sous forme de tours simultanés.

À chaque tour, les deux joueurs doivent sélectionner secrètement une attaque parmi les quatre attaques disponibles pour leur créature.

Le joueur 1 utilise les touches :

- W
- A
- S
- D

Le joueur 2 utilise les touches :

- Flèche du haut
- Flèche de gauche
- Flèche du bas
- Flèche de droite

Une fois les deux choix effectués, les attaques sont révélées simultanément.

Les deux attaques doivent être calculées avant d'être appliquées. Une créature éliminée durant un tour inflige tout de même les dégâts de l'attaque qu'elle avait sélectionnée.

Les dégâts sont ensuite appliqués et les points de vie des deux créatures sont mis à jour.

Après chaque tour, les informations pertinentes du combat doivent être affichées.

Fin du combat

Le combat prend fin lorsqu'une des deux créatures possède zéro point de vie ou moins.

Le gagnant doit être annoncé clairement.

Les créatures doivent demeurer dans la collection après le combat.

Architecture orientée objet

La classe de base représentant une créature doit être abstraite.

Au moins une méthode virtuelle pure doit être utilisée.

Les différents types de créatures doivent utiliser l'héritage et le polymorphisme.

L'utilisation d'une seule classe *Creature* contenant un attribut *type* ne satisfait pas les exigences du travail.

Les collections doivent contenir des objets manipulés polymorphiquement.

Une collection de type *vector Creature**, une solution équivalente utilisant le polymorphisme est attendue.

Les effets propres à chaque type doivent être implémentés à l'aide de l'héritage et du polymorphisme.

Une série de conditions (*if*, *else if*, *switch*) basées sur le type d'une créature ne satisfait pas cette exigence.

Contraintes

- Vous devez utiliser au moins une classe abstraite.
- Vous devez utiliser au moins une méthode virtuelle pure.
- Vous devez utiliser au moins une collection contenant des objets manipulés polymorphiquement.
- Chaque type de créature doit être représenté par une classe distincte.
- Le programme doit fonctionner entièrement en console.

Pondération

Le travail correspond à **25% de votre note finale**.

- Création de créatures
 - Création d'une créature personnalisée : 5 pts
 - Validation des statistiques : 5 pts
 - Création des différents types de créatures : 10 pts

Sous-total : 20 pts

- Gestion des créatures
 - Affichage des créatures : 5 pts
 - Modification d'une créature : 5 pts
 - Suppression correcte d'une créature : 5 pts

Sous-total : 15 pts

- Système d'attaques
 - Gestion des quatre attaques par type : 10 pts
 - Affichage adéquat des attaques : 5 pts

Sous-total : 15 pts

- Combat
 - Sélection des combattants : 5 pts
 - Sélection secrète des attaques : 10 pts
 - Calcul et application des dégâts : 10 pts
 - Détermination correcte du gagnant : 5 pts

Sous-total : 30 pts

- Architecture orientée objet
 - Utilisation correcte de l'héritage : 5 pts
 - Utilisation correcte du polymorphisme : 5 pts
 - Classe abstraite et méthodes virtuelles : 5 pts
 - Gestion adéquate des constructeurs et destructeurs : 5 pts

Sous-total : 20 pts

[20 + 15 + 15 + 30 + 20 = 100 points]

Je me permets de retirer jusqu'à **20%** de la note obtenue selon la qualité du code, son efficacité, sa lisibilité, sa modularité ainsi que le respect des conventions de programmation vues en classe.

Entraide

L'entraide est permise et encouragée. Vous pouvez vous entraider sur la logique des scripts et sur l'explication des concepts vus en classe. Par contre, il est interdit de partager son code, de copier d'aucune manière le code de quelqu'un d'autre ou de remettre un travail effectué par quelqu'un d'autre. Il y a une différence très marquée entre l'entraide et le plagiat.

Remise

- Un dossier comprenant tout votre projet en format .zip.

La remise s'effectue au plus tard le 1 juillet à 23 h 59. Tout retard entraîne une pénalité sur la note de 5%, jusqu'à un maximum de 3 jours, après quoi une note d'échec est attribuée.